

Automatic Dog Feeder artefact

Link to YouTube video link explaining how the system works: <https://youtu.be/B0yXcGgZ2DM>

Link to GitHub file with source code and secrets file: <https://github.com/Dmatt547/Automatic-Dog-Feeder/tree/main>

Feeding pets, especially dogs can be a very challenging task for pet owners who are often away from home due to work commitments or other personal reasons, ensuring their pets receive their meals at correct intervals and in the correct quantities is crucial for the wellbeing and overall health of the dogs. This project aims at addressing these challenges by developing an automatic dog feeder that can dispense food based on the weight in the dog bowl. If the weight in the dog bowl goes below a certain amount, the dog feeder will dispense the correct amount of food after certain intervals.

Automatic dog feeders are available on the market already; however, they lack some special features that mine has. They lack the ability to measure and dispense food based on the remaining food weight in the bowl. This project integrates weight management and a real time data display with a notification system letting the owner know when their pet has been fed. This system is more user friendly, and it is able to integrate well with the user in their day-to-day life.

The primary problem addresses by this project is the need for a more reliable and automated way to feed dogs/pets as well as ensuring they receive the correct amount of food at the right time intervals during the day when the owner is not present. This Automatic Dog Feeder also implements a temperature and humidity sensor, allowing the user to monitor the conditions of the room where the food is. If the temperature goes too high or too low, the user may want to make sure that the dog is staying in a more comfortable environment, and also ensure that the food doesn't go mouldy due to extreme heat.

The requirements of this system are as follows:

- Measure the weight of the food in the bowl.
- Automatically dispense food when the weight in the bowl goes below a specified threshold at the correct time intervals.
- Display real time weight, temperature and humidity data in a simple and easy to use UI.
- Allow the user to receive notifications on their phone to let them know when their dog has been fed.

Design principles:

- Fault tolerance:
 - **Error handling:** The DHT sensor is able to check if the data read has failed, if so it prints an error message and exits the loop to prevent further processing of incorrect data. Also, during the load cell initialization, if the tare operation times out, the code prints an error message and enters an infinite loop to prevent further execution until the issue is finally resolved.
 - **Crash recovery:** The system is designed to restart crucial tasks in case of a crash, such as reinitialising the load cell and reestablishing the Wi-Fi connection if needed.

- **Hand recovery:** The system is able to use timeouts and checks to ensure that the system doesn't hang when trying to connect the Wi-Fi. It waits for the Wi-Fi connection to be established with a delay and checks for connectivity status.
- **Hardware-aware design:**
 - **Speed mismatch:** The code uses interrupts and timed updates to handle the speed mismatch between the hardware components and the MCU.
 - **Time sensitive interactions:** The code uses precise timing for servo movement and updates to ensure that it moves smoothly and in the correct position.
 - **Non-responsive hardware:** The code integrates timeouts for Wi-Fi connections and server responses to handle non-responsive hardware or network issues.
- **Performance:**
 - **Efficient process:** The code avoids heavy duty operations within the main loop, ensuring real time performance is maintained at a good level.
 - **Code optimizations:** The code uses variables to store commonly used values to avoid redundant operations.
- **Maintainability:**
 - **Remote fixes:** The system is designed to be connected remotely to allow for easy fixes in the code, allowing for remote monitoring and updates through IoT frameworks.

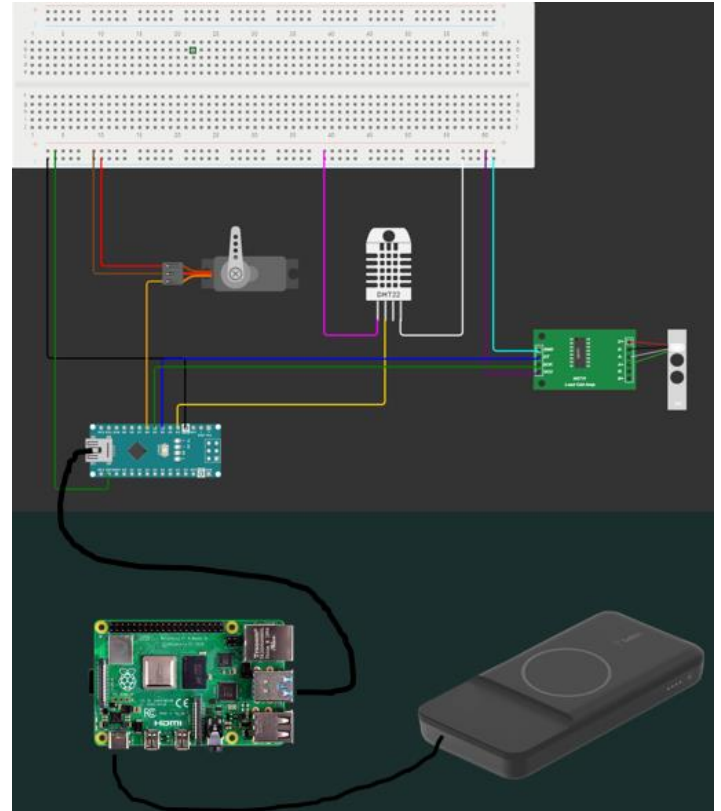
Hardware architecture:

- Arduino Nano 33 IoT: Controls the sensors and communicates data to the Raspberry Pi.
- Raspberry Pi: Hosts the user interface and processes data received from the Arduino.
- Load Cell Sensor: Measures the weight of the food in the bowl.
- DHT11 Humidity and Temperature Sensor: Monitors environmental conditions.
- Servo Motor: Controls the food dispensing mechanism.
- Breadboard and Jumper Wires: Used for connecting the components together.

Network architecture:

The Arduino Nano 33 IoT collects the sensor data from the load cell and the DHT11 sensor. It then sends the data over a serial connection to the Raspberry Pi. The Raspberry Pi processes the data and displays it on a Tkinter GUI.

The Raspberry Pi is displayed over an application called RealVNC Viewer. This application uses the IP address of the Pi to remotely connect to it and mirror its display onto my computer screen. This is very



beneficial as it means that I don't have to connect the Raspberry Pi to a monitor, keyboard and mouse when I want to write code on it.

The user of this Automatic Dog Feeder will be able to view the real time data through the mirrored display, making it easier to monitor their dogs food intake.

Software architecture:

- Arduino Code: Handles sensor data collection and serial communication, as well as IFTTT protocol.
- Raspberry Pi Code: Uses Python with the Tkinter library to display real-time data.

Testing approach:

To ensure accurate weight measurements, I conducted multiple extensive testing and calibration of the load cell. Initially I had faced several issues with fluctuating load cell readings, which required several adjustments and trials to stabilise the readings. This calibration process involved the following steps:

- Initial calibration
 - Set up the load cell and performed a tare operation on the zero scale.
 - Applied known weights and adjusted the calibration factor until the readings matched the actual weights.
- Stabilizing readings
 - To address the fluctuating readings, I experimented with different mounting configurations and finally stuck to one specific mounting option with 3D printed parts to ensure a precise fit with the load cell.

Servo motor and funnel testing:

- Incremental testing
 - Conducted test with the servo motor opening at 5 degree increments to find the best opening angle to allow for the best and most controlled flow rate of the food.
 - Measure the amount of food dispensed at each angle and identified the relationship between the servo angle and the quantity of the food being dispensed.
- Average readings
 - For each servo angle, I ran the tests and recorded the readings.
 - I then calculated the average amount of food dispenses for each angle.
- Error calculation
 - I compared the average readings to the target weight.
 - I then determined the percentage error for each test to understand how much the actual dispensed weight deviated from the target weight.

Adjusting for accurate dispensing:

- Setting target weight

- I adjusted the target weight in the code to account for the percentage error observed during my testing.
- Set the target weight to be the actual desired weight minus the percentage error. This ensured that the servo started closing before reaching the actual target weight.
- Timing adjustments
 - I incorporated a delay to account for the time that it takes for the servo to fully close. This was needed because the servo motor has a slight delay in its movement, and starting the closing process before reaching the exact weight helped achieve the accurate dispensing.

Final testing:

- After making the adjustments, I was able to conduct my final testing to validate the systems performance.
- I ran multiple complete cycles of the automatic dog feeder to ensure that the load cell readings, servo motor operation, and food dispensing were all functioning correctly.
- Validation was conducted.
 - I ensured that the weight of the food dispensed closely matched the target weight in the repeated trials. There were several trials where the weight dispensed was out by +-20 grams, however this is due to the inconsistency in the food being dispensed. Due to the narrow funnel opening, the food being dispensed was getting caught for a split second, affecting the flow rate out of the funnel and thus the weight being dispensed was not the same all the time. This can be fixed with a different design in the dispensing process. In a future design, it might be worth changing the dispensing process to incorporate a tipping movement instead of a latch that opens and closes. This would help the food be dispensed with more control.

User manual:

1. Loading the Correct Code onto the Arduino

- Ensure that the correct coding file is loaded onto the Arduino Nano 33 IoT. Use the Arduino IDE to upload the provided code to the Arduino.

2. Setting Up Remote Connection

- RealVNCViewer Setup:
 1. Install RealVNCViewer on your computer.
 2. Use RealVNCViewer to remotely connect to the Raspberry Pi to display the Tkinter GUI.
 3. Insert the username (danie) and the password, which is the IP address of the Raspberry Pi.

3. Ensuring Power Supply

- Power the System:
 1. Ensure that the system has power. Use a portable battery bank placed inside the main container for portability.

2. Verify that the Raspberry Pi and Arduino are connected via the serial connection.

4. Troubleshooting Wi-Fi Connection

- Connecting Arduino to Wi-Fi:
 1. There may be an issue with getting the Arduino to connect to the Wi-Fi network at first.
 2. If the Arduino doesn't connect, unplug and then plug in the USB cable to restart the Arduino.
 3. Attempt to gain the Wi-Fi connection again. This may take a few attempts before the load cell data is displayed on the Tkinter GUI.

Introduction

This guide will help you assemble the main structure of the automatic dog feeder, including the housing, components, and connections.

Materials Needed

- Timber plank
- Metal beam (40 cm in height)
- Funnel
- Servo motor
- Flat piece of material (for the servo arm)
- Load cell
- Two blocks to hold the load cell
- Container (for the food, acting as the dog bowl)
- Breadboard
- Arduino Nano 33 IoT
- Raspberry Pi
- Battery pack
- Container for electrical components
- DHT11 humidity and temperature sensor
- Drill, bolts and nuts.

Assembly Instructions

Step 1: Preparing the Base

1. Timber Plank: Use a sturdy piece of timber as the base to house all the main components.
2. Metal Beam: Attach a metal beam at a height of 40 cm above the timber plank. This will support the funnel.

Step 2: Mounting the Funnel

1. Funnel: Bolt the funnel to the top of the metal beam. This will hold the dog food before it gets dispensed.
2. Servo Motor: Mount the servo motor upside down at the bottom of the funnel to control the hatch.
3. Servo Arm: Attach a flat piece of material to the servo arm. This will act as a barrier to control the flow of food from the funnel.

Step 3: Installing the Load Cell

1. **Wooden Blocks:** Place two wooden blocks on the timber plank to elevate the load cell.
2. **Load Cell:** Sandwich the load cell between the two wooden blocks. Ensure it is securely mounted.
3. **Food Container:** Attach a container on top of the load cell. This container will act as the dog bowl and collect the dispensed food.

Step 4: Setting Up the Electrical Components

1. **Container for Electronics:** Use a separate container to house the breadboard, Arduino Nano 33 IoT, Raspberry Pi, and battery pack.
2. **Drill Holes:** Drill holes in the side of the container to pass through the wires for the servo, load cell, and temperature and humidity sensor.
3. **Sensor Placement:** Mount the DHT11 sensor outside the container to ensure accurate temperature and humidity readings.

Step 5: Wiring and Connections

1. **Breadboard:** Set up the breadboard inside the electronics container and make the necessary connections for the Arduino, Raspberry Pi, servo motor, load cell, and DHT11 sensor.
2. **Power Supply:** Connect the battery pack to power the system.
3. **Cable Management:** Ensure all cables are neatly arranged and securely connected to avoid interference when closing the container.

Final Setup

1. **Secure All Components:** Ensure that all components are securely mounted and all connections are properly made.
2. **Test the System:** Power on the system and perform initial tests to ensure everything is functioning correctly.
3. **Adjustments:** Make any necessary adjustments to the positioning of the components for optimal performance.

Conclusion:

Working on this project has been a very enriching and exciting experience, providing me with very practical insights into the world of embedded systems and real-time data processing. One of the main challenges I faced when building my project was ensuring reliable communication between the Arduino and the Raspberry Pi. I also faced challenges of getting the correct amount of food to be dispensed every time. However, with extensive testing and robust error handling, I was able to overcome these challenges. If I was given a second chance, I would explore integrating wireless communication to reduce the wiring complexity and improve the system's scalability. I would also integrate a new system to dispense the food with a tilting system to accurately dispense the food with more precision and control.

Issues faced:

- **Sensor interference:** Dealing with noise and interference in the Load Cell sensor readings.
- **Servo motor reliability:** Ensuring that there was a consistent and reliable operation of the servo motor.

- Dispensing accuracy: There were several trials where dispensing was out by ± 20 grams. This inconsistency was due to the narrow funnel opening, causing the food to get caught for a split second and affecting the flow rate. This issue led to variations in the dispensed weight of the food. In a future design, I could address this issue by changing the dispensing mechanism to incorporate a tipping movement instead of a latch that opens and closes. This would significantly help the flow rate stay more consistent and controlled, and thus having an overall effect on the accuracy of the food being dispensed. By addressing these issues and making the suggested improvements, the automatic dog feeder can become more reliable, accurate and user friendly.